
Prompt Forge Testing Policy

1. Introduction to the Project

Prompt Forge is a web-based application that helps individuals and organizations craft, test, and share effective prompts for Large Language Models (LLMs). It features prompt evaluation, A/B testing, versioning, and a marketplace for premium and public prompts. Built for scalability, usability, and extensibility, Prompt Forge supports a wide range of users including prompt engineers, educators, and enterprises.

2. Testing Policy Introduction

This policy establishes the standards and processes for testing the Prompt Forge platform. It focuses specifically on unit testing, integration testing, and end-to-end (E2E) testing. The goal is to ensure the system is robust, maintainable, and reliable through automated test coverage and CI/CD validation.

3. Unit and Integration Testing Policy

3.1 Unit Testing Policy

3.1.1 Objective

Unit testing is the first line of defense against software defects. It ensures that the smallest pieces of business logic—classes, functions, and methods—work correctly in isolation. The goals of unit testing in Prompt Forge are:

- Validate business logic against expected behavior
- Detect issues early in the development cycle
- Enable fearless refactoring
- Improve long-term maintainability
- Provide working examples for new contributors

3.1.2 Technologies Used

We use two parallel setups for unit testing in the frontend and backend.

Background (Java):

We use a robust and modern Java-based test stack to write expressive, maintainable unit tests:

- **JUnit 5** – Modern and extensible test framework with lifecycle and parameterized test support
- **Mockito** – Allows mocking of services, DAOs, and components to isolate the unit under test
- **Spring Boot Test** – Lightweight support for testing with Spring context when required
- **AssertJ** – Fluent assertions for expressive and readable test expectations
- **Maven Surefire Plugin** – Runs unit tests and generates XML reports for CI analysis

```
@Test
void purchase_ShouldSuccessfullyPurchaseAndClearCart() throws Exception {
    String email = "test@example.com";
    CartItemDTO dto = mock(classToMock:CartItemDTO.class);
    when(dto.getPromptId()).thenReturn(promptId);
    List<CartItemDTO> prompts = Collections.singletonList(dto);

    when(userService.getUserIdByEmail(email)).thenReturn(userId);
    when(promptStoreService.purchasePrompt(promptId, userId)).thenReturn(mock(classToMock:PromptPurchase.class));
    doNothing().when(cartItemRepository).deletebyUserID(userId);

    assertDoesNotThrow(() -> service.purchase(email, prompts));

    verify(promptStoreService).purchasePrompt(promptId, userId);
    verify(cartItemRepository).deletebyUserID(userId);
}
```

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.fiveOps.promptforge.analytics.controller		100%		n/a	0	4	0	6	0	4	0	1
com.fiveOps.promptforge.analytics.services		100%		n/a	0	5	0	12	0	5	0	1
com.fiveOps.promptforge.authentication.controller		61%		33%	5	12	31	88	0	6	0	1
com.fiveOps.promptforge.authentication.service		58%		60%	6	15	23	57	4	10	0	1
com.fiveOps.promptforge.cart.controller		92%		100%	0	8	2	41	0	6	0	1
com.fiveOps.promptforge.cart.service		100%		100%	0	11	0	45	0	9	0	1
com.fiveOps.promptforge.dashboard.controller		95%		78%	10	28	6	89	0	5	0	1
com.fiveOps.promptforge.dashboard.services		100%		83%	1	11	0	12	0	8	0	1
com.fiveOps.promptforge.integration.controller		96%		n/a	0	7	4	122	0	7	0	1
com.fiveOps.promptforge.ml.controller		100%		n/a	0	4	0	21	0	4	0	1
com.fiveOps.promptforge.payments.controller		59%		66%	2	9	34	71	0	6	0	1
com.fiveOps.promptforge.payments.service		88%		64%	10	32	32	223	0	15	0	4
com.fiveOps.promptforge.prompts.controller		97%		89%	6	41	4	108	0	13	0	1
com.fiveOps.promptforge.prompts.service		91%		87%	8	58	16	245	3	38	0	3
com.fiveOps.promptforge.promptstore.controller		100%		100%	0	17	0	35	0	16	0	1
com.fiveOps.promptforge.promptstore.service		100%		100%	0	38	0	73	0	28	0	1
com.fiveOps.promptforge.S3Bucket.service		95%		100%	0	5	2	19	0	3	0	1
com.fiveOps.promptforge.testground.controller		89%		81%	10	41	27	206	0	12	0	1
com.fiveOps.promptforge.user_profile.controller		100%		100%	0	35	0	84	0	20	0	1
com.fiveOps.promptforge.user_profile.service		80%		87%	21	60	16	125	14	33	0	1
com.fiveOps.promptforge.util.service		14%		n/a	1	2	6	7	1	2	0	1
Total		832 of 7 266		66 of 386	80	443	203	1 689	22	250	0	26

Frontend (React):

- **Cypress Component Testing** – Used to test isolated React components with simulated props, states, and interactions in a browser environment

Cypress component tests provide a browser-based rendering layer that mimics real usage closer than pure logic-based tests, giving us confidence in our reusable UI pieces like buttons, modals, form elements, and layout containers.

Component tests run in headless mode in CI and in browser mode locally for debugging.

TERMINAL

Duration: 5 seconds
Spec Ran: Tabs.cy.tsx

Test Results Summary:
Total Tests: 53
Passing: 52
Failing: 1
Skipped: 0
Duration: 30095ms

=====
(Run Finished)

Spec	Tests	Passing	Failing	Pending	Skipped
✓ Badge.cy.tsx	00:02	6	6	-	-
✓ Button.cy.tsx	00:02	6	6	-	-
✓ Card.cy.tsx	00:02	5	5	-	-
✓ Image.cy.tsx	00:01	6	6	-	-
✗ Input.cy.tsx	00:06	4	3	1	-
✓ Label.cy.tsx	00:03	6	6	-	-
✓ Select.cy.tsx	00:03	7	7	-	-
✓ StandardPromptCard.cy.tsx	00:02	5	5	-	-
✓ Tabs.cy.tsx	00:05	8	8	-	-

frontend 3.3s 1 6 6 0

Label Component

2.7s 6 6

- ✓ renders with text content 823ms
- ✓ associates with form inputs via htmlFor 711ms
- ✓ applies custom className 470ms
- ✓ supports HTML label attributes 240ms
- ✓ renders complex content 210ms
- ✓ handles click events 267ms

Test files are located under `src/test/java` and mirror the structure of `src/main/java`. Each class under test has a corresponding test class with the same name suffixed by `Test`.

Example layout:

`src/`

```
|— main/java/com/promptforge/user/UserService.java
|
|— test/java/com/promptforge/user/UserServiceTest.java
```

Each test class:

- Initializes the class under test
- Mocks dependencies using Mockito
- Uses assertions to validate expected outcomes

3.1.4 Automation

Unit tests are triggered automatically on every code push and pull request.

GitHub Actions runs the workflow defined in `.github/workflows/test.yml`. Steps include:

- Checking out the code
- Setting up the JDK
- Caching Maven dependencies
- Running `mvn clean test`

Tests must pass for the CI pipeline to proceed.

3.1.5 Outcome

- All tests pass: the PR continues to next phase
- One or more tests fail: the CI fails, logs are reported, and the PR is blocked

3.1.6 Reporting

- Test results are stored as XML reports in `target/surefire-reports/`
- These are parsed by GitHub Actions for summary output
- Includes JaCoCo HTML dashboards for visual test history

3.2 Integration Testing

3.2.1 Objective

Integration tests validate that multiple components work together correctly. Unlike unit tests, they involve actual Spring Boot context, real or embedded databases, and interaction between services, controllers, and repositories.

Goals include:

- Confirm services integrate correctly with persistence layers
- Ensure data flows between controllers → services → repositories
- Test real business logic paths using realistic scenarios

3.2.2 Technologies Used

- **JUnit 5 with Spring Boot Test** – Brings up Spring context for real integration
- **Mockito** – For stubbing side-effects if required
- **Maven Failsafe Plugin** (*planned*) – For executing integration tests separately

3.2.3 Test Environment

- Tests run within GitHub Actions using configured CI runners

3.2.4 Automation

- GitHub Actions configures test environment and runs integration tests
- Integration tests are marked using `@SpringBootTest` and categorized appropriately

3.2.5 Outcome

- Success: tests pass and app is ready for E2E testing
- Failure: errors are logged, merge is blocked until fixed

3.2.6 Reporting

- Maven logs test output to the CI pipeline
- Future integration: Allure reports with timeline-based failure views

```
9717 [INFO]
9718 [INFO] Tests run: 481, Failures: 0, Errors: 0, Skipped: 0
9719 [INFO]
9720 [INFO]
9721 [INFO] --- jacoco:0.8.11:report (report) @ project ---
9722 [INFO] Loading execution data file /home/runner/work/Prompt-Forge/Prompt-Forge/backend/pr
9723 [INFO] Analyzed bundle 'project' with 43 classes
9724 [INFO] -----
9725 [INFO] BUILD SUCCESS
9726 [INFO] -----
9727 [INFO] Total time: 01:48 min
```

4. End-to-End (E2E) Testing

4.1 Objective

End-to-end tests simulate real user interactions with the application to validate the entire flow, from frontend UI to backend APIs and database operations. E2E tests catch issues that unit and integration tests may miss due to incomplete environments or mocks.

Goals:

- Validate UI behavior and page interactions
- Confirm backend integration through real API calls
- Ensure reliability of critical user flows

4.2 Framework

- **Cypress** is used as the main E2E framework due to its fast execution and powerful selector engine
- Includes UI interaction simulation, network request monitoring, and assertion capabilities

4.3 Testing Environment

- Application is served locally
- Cypress connects to backend services through configured ports

4.4 Automation

- Tests are triggered by GitHub Actions when a PR is opened or updated
- Cypress executes test flows such as login, dashboard access, prompt creation, and feedback workflows

- Screenshots and logs are captured automatically on failure

4.5 Outcome

- All test cases pass: build is flagged as stable
- Any test fails: CI fails, logs and screenshots are saved, and the PR is blocked until fixed

4.6 Reporting

- Cypress reports are uploaded as CI artifacts or HTML files
- Screenshots and videos are retained per test run
- Future integrations may include Slack notifications and Allure dashboards

5. Code Coverage

- **Minimum 70% coverage** required for all service layers and critical components
- **Frontend:** Cypress with coverage instrumentation
- **Backend:** JaCoCo integrated via Maven plugin
- Thresholds are enforced at CI level – PRs are blocked if coverage drops below minimum

project													
project													
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes	
com.fiveOps.promptforge.user_profile.service		52%		37%	39	56	63	137	14	29	0	1	
com.fiveOps.promptforge.testground.controller		29%		50%	4	6	19	27	1	3	0	1	
com.fiveOps.promptforge.cart.service		64%		66%	3	12	18	56	1	9	0	1	
com.fiveOps.promptforge.user_profile.controller		68%		50%	15	26	13	45	5	16	0	1	
com.fiveOps.promptforge.prompts.controller		82%		73%	9	28	12	57	0	11	0	1	
com.fiveOps.promptforge.cart.controller		90%		75%	1	8	2	37	0	6	0	1	
com.fiveOps.promptforge.prompts.service		95%		50%	3	29	3	68	2	28	0	2	
com.fiveOps.promptforge.dashboard_dash_controller		96%		83%	2	9	2	41	0	3	0	1	
com.fiveOps.promptforge.authentication.service		95%		100%	1	9	0	29	1	6	0	1	
com.fiveOps.promptforge.authentication.controller		99%		50%	1	5	0	43	0	4	0	1	
com.fiveOps.promptforge.promptstore.service		100%		100%	0	32	0	62	0	27	0	1	
com.fiveOps.promptforge.promptstore.controller		100%		100%	0	16	0	29	0	15	0	1	
com.fiveOps.promptforge.analytics_ana_services		100%		n/a	0	5	0	11	0	5	0	1	
com.fiveOps.promptforge.dashboard_dash_services		100%		100%	0	7	0	8	0	6	0	1	
com.fiveOps.promptforge.analytics_ana_controller		100%		n/a	0	4	0	6	0	4	0	1	
Total	619 of 2 887	78%	63 of 160	60%	78	252	132	656	24	172	0	16	

6. Required Links

6.1 Links to Tests

- **Frontend Unit & E2E:** frontend/src/**/* .test.tsx, frontend/cypress/e2e/* .cy.ts
- **Backend Unit Tests:** backend/project/src/test/.../**.java
- **Backend Integration Tests:** backend/project/src/test/**/.../**.java

7. Testing Tools

- **Unit:** JUnit 5, Mockito, AssertJ, Maven Surefire
- **Integration:** Spring Boot Test, Failsafe Plugin (planned)
- **E2E:** Cypress
- **Coverage:** JaCoCo, Cypress Coverage Plugin, Codecov

8. Test Reports

8.1 GitHub Actions

- All test phases (unit, integration, E2E) produce summary output
- Failures are logged in the Actions tab with logs, stack traces, and job metadata
- Artifacts include raw reports, screenshots, and coverage data